

Apprentissage statistique : modélisation décisionnelle et apprentissage profond (RCP209)

Réseaux de neurones récurrents

Nicolas Audebert

`nicolas.audebert@lecnam.net`

`http://cedric.cnam.fr/vertigo/Cours/ml2/`

Département Informatique
Conservatoire National des Arts & Métiers, Paris, France

17 mai 2023

Plan du cours

1 Réseaux de neurones récurrents

- Modélisation décisionnelle sur des séquences
- Réseaux récurrents

2 Optimisation des réseaux récurrents

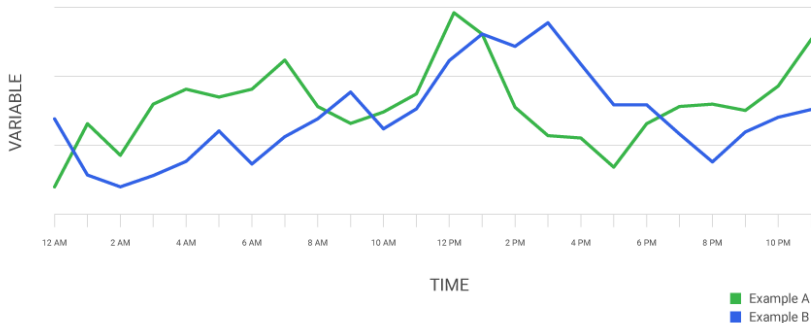
Traitement de séquences

Définition

Une séquence est une observation $\{x_t\}$ pour $t \in \{1; T\}$, avec $x_t \in \mathbb{R}^d$.

En général, l'observation au pas de temps x_t dépend de $x_{t'}$ pour $t' \leq t$ (causalité).

Exemples : texte, séries temporelles, ADN, logs, etc.

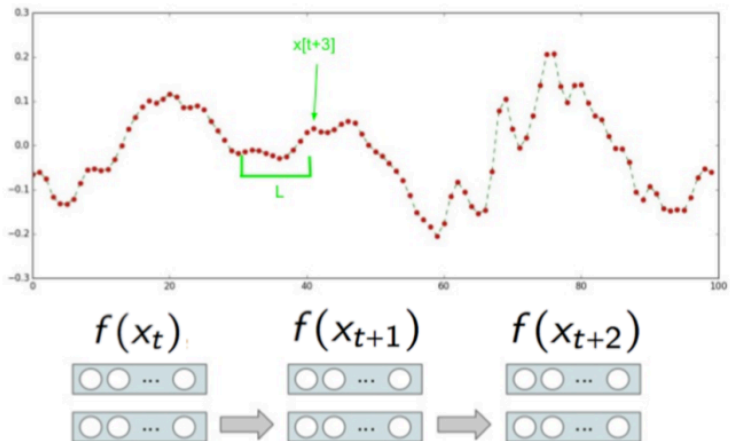


Modélisation décisionnelle sur des séquences

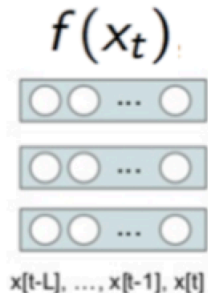
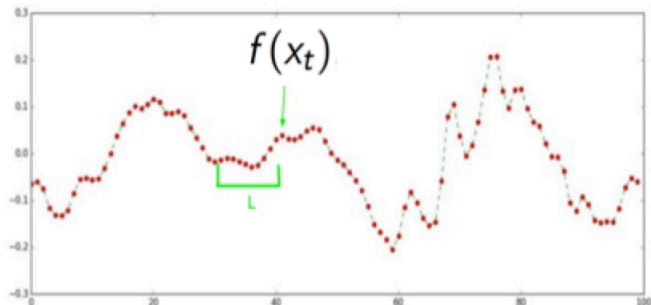
On cherche une fonction f permettant de prédire y_t à partir de x_t .

Option 1 : perceptron multi-couche

Comme pour les SVM ou les forêts aléatoires, on peut traiter une séquence par un MLP sur une fenêtre de taille fixe L .



Limites des modèles entièrement connectés

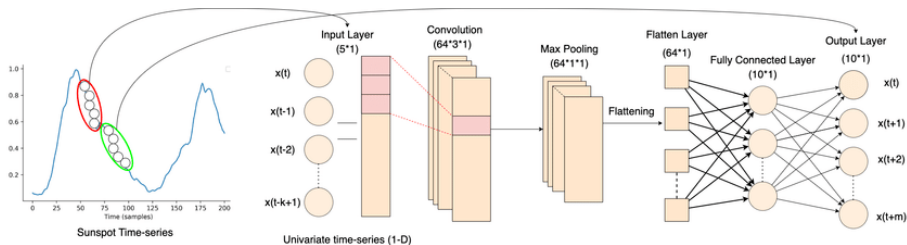


- $\ominus$ Si le contexte temporel à prendre en compte est grand, $L \nearrow \Rightarrow$ nombre de paramètres $\nearrow$,
- $\ominus$ Les prédictions sont indépendantes à chaque pas de temps x_t (pas de "mémoire" de y_{t-1}),
- $\ominus$ Impossible de traiter des séquences de longueur variable, sauf à les redécouper en fenêtre de taille fixe.

Réseaux convolutifs sur des séquences

Option 2 : CNN

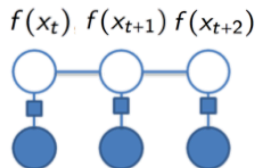
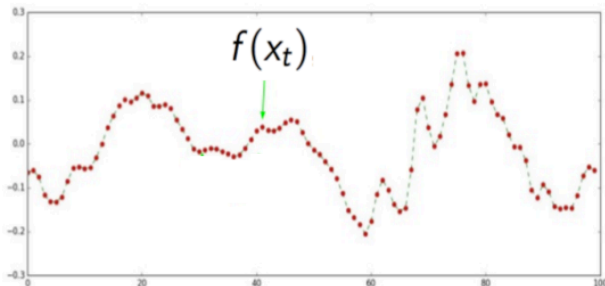
Modèle convolutif à noyaux unidimensionnels (convolution 1D) sur une fenêtre de taille fixe L .



- $\oplus$ Moins de paramètres que les modèles entièrement connectés, extrait l'information locale
- $\ominus$ Ne peut pas traiter des séquences de longueur variable à cause de la dernière couche entièrement connectée

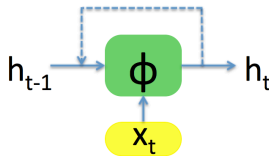
Prédiction structurée : modèles graphiques

- Prédiction structurée : modélisation explicite des liens entre $f(x_t)$ et $f(x_{t'})_{t' \leq t}$
- Chaînes de Markov (modèle génératif $P(x, y)$), Conditional Random Fields (CRF)



- $\oplus$ Gère des séquences de longueur variable L
- $\ominus$ Limités aux prédicteurs linéaires
- $\ominus$ Procédure d'inférence complexe (solution extraite non-tractable)
 - Hypothèse de Markov : $f(x_T|x_t, t \leq T) = f(x_T|x_{T-1})$
 - le présent ne dépend que d'un nombre limité de valeurs passées

Cellule récurrente



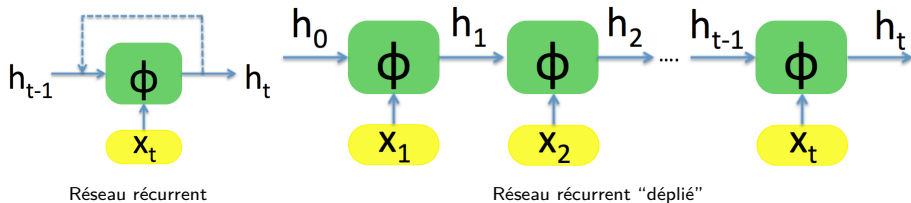
Réseau récurrent

- Entrée : séquence $\{\mathbf{x}_t\}_{t \in \{1; T\}}$, $\mathbf{x}_t \in \mathbb{R}^d$
- État interne du RNN : $\{\mathbf{h}_t\}_{t \in \{1; T\}}$, $\mathbf{h}_t \in \mathbb{R}^l$
- La cellule récurrente est définie par : (ELMAN 1990)

$$\mathbf{h}_t = \phi_t(\mathbf{x}_t, \mathbf{h}_{t-1})$$

- Boucle récursive : $\mathbf{h}_t$ dépend de l'observation présente $\mathbf{x}_t$ et de l'état interne précédent $\mathbf{h}_{t-1}$
- $\mathbf{h}_t$ modélise la mémoire du réseau (historique jusqu'au pas de temps t)
- Dans les RNN, la fonction $\phi_t = \phi$ est **identique** ("partagée") pour tous les pas de temps t .

Cellule récurrente



- Entrée : séquence $\{x_t\}_{t \in \{1; T\}}$, $x_t \in \mathbb{R}^d$
- État interne du RNN : $\{h_t\}_{t \in \{1; T\}}$, $h_t \in \mathbb{R}^l$
- La cellule récurrente est définie par : (ELMAN 1990)

$$h_t = \phi_t(x_t, h_{t-1})$$

- Boucle récursive : h_t dépend de l'observation présente x_t et de l'état interne précédent h_{t-1}
 - h_t modélise la mémoire du réseau (historique jusqu'au pas de temps t)
- Dans les RNN, la fonction $\phi_t = \phi$ est **identique** ("partagée") pour tous les pas de temps t .

Réseau récurrent

Cellule récurrente : $\mathbf{h}_t = \phi(\mathbf{x}_t, \mathbf{h}_{t-1})$

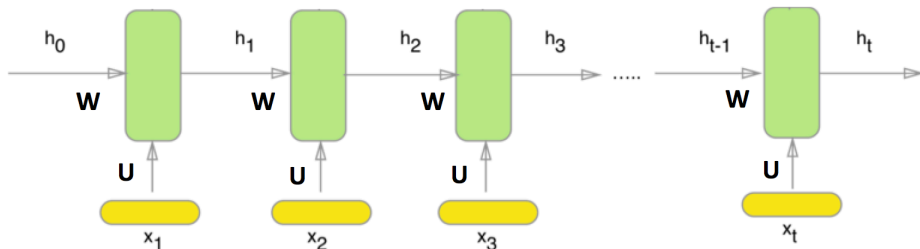
Définition d'un réseau récurrent

- On choisit pour ϕ une projection linéaire entre $\mathbf{x}_t$ et $\mathbf{h}_{t-1}$, c'est-à-dire des couches entièrement connectées :

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

Si l'on choisit la dimension de l'état caché $\mathbf{h}$ égale à l , alors :

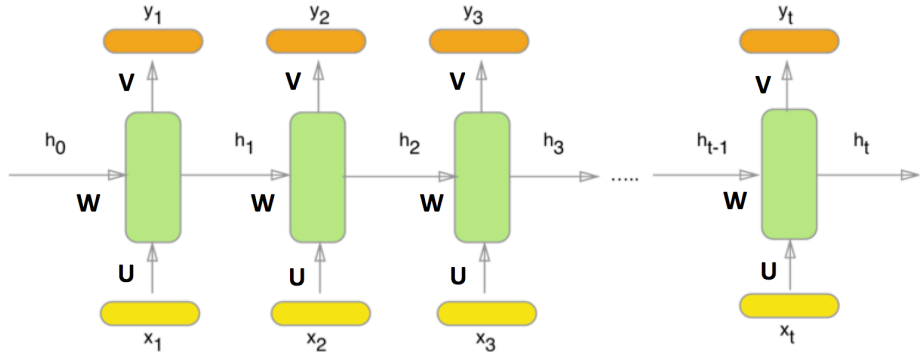
- $\mathbf{U}$ est une matrice $(l \times d)$, $\mathbf{W}$ est une matrice $(l \times l)$
- $\mathbf{b}$ est un vecteur de biais de longueur l
- f est une fonction d'activation non-linéaire, généralement $\tanh$



Prédiction par un réseau récurrent

Pour la prédiction de la sortie, on ajoute une couche entièrement connectée entre l'état interne et la sortie :

- À chaque pas de temps t , la sortie du RNN est $y_t = f'(Vh_t + b_y)$
 - f' peut être l'identité (régression) ou une activation softmax (classification)

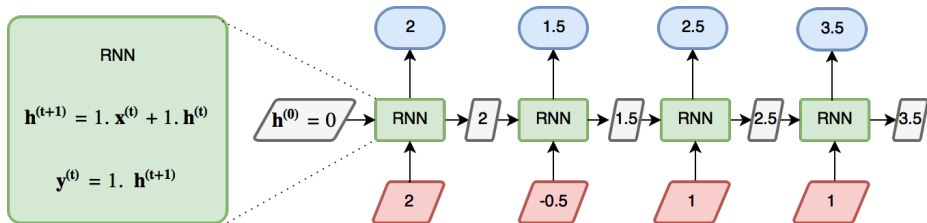


Capacités de modélisation des RNN

- Rappel : les réseaux entièrement connectés sont des approximateurs universels de fonction
- Quels résultats a-t-on concernant les transformations entre $\{\mathbf{x}_t\}_{t \in \{1; T\}}$ et $\{\mathbf{y}_t\}_{t \in \{1; T\}}$?
 - Les RNN sont approximateurs universels de programme SIEGELMANN et SONTAG 1995
 - Les RNN peuvent approximer n'importe quelle fonction calculable ($\sim$ machine de Turing)
- Les RNN peuvent approcher n'importe transformation séquence à séquence mesurable HAMMER 2000

Exemple 1 : calcul d'une somme par réseau récurrent

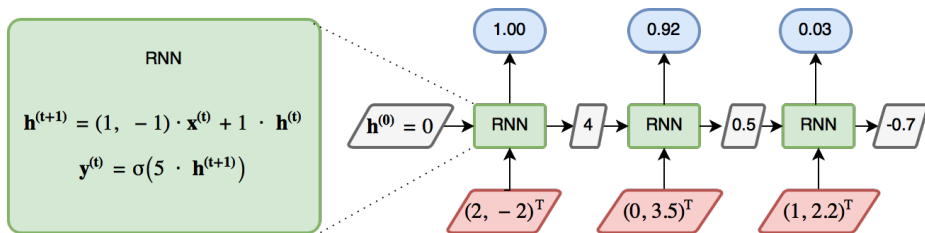
Objectif : calculer $y_t = \sum_{t'=1}^t x_{t'}$



Exemple 2 : comparaison de la somme selon deux dimensions par réseau récurrent

Objectif : calculer $y_t = \sum_{t'=1}^t x_{1,t'} > \sum_{t'=1}^t x_{2,t'}$

- État interne : calculer $\text{dim}_1 - \text{dim}_2$ puis sommer



Plan du cours

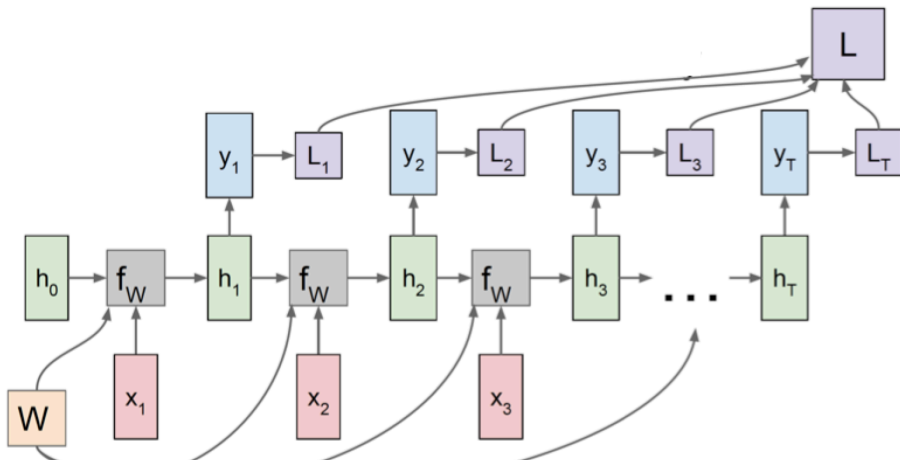
- 1 Réseaux de neurones récurrents
 - Modélisation décisionnelle sur des séquences
 - Réseaux récurrents
- 2 Optimisation des réseaux récurrents

Entraînement des réseaux récurrents

Comme pour les réseaux de neurones classiques, on compare la sortie prédite $\{\mathbf{y}_t\}_{t \in \{1; T\}}$ à une vérité de terrain (supervision) $\{\mathbf{y}_t^*\}$.

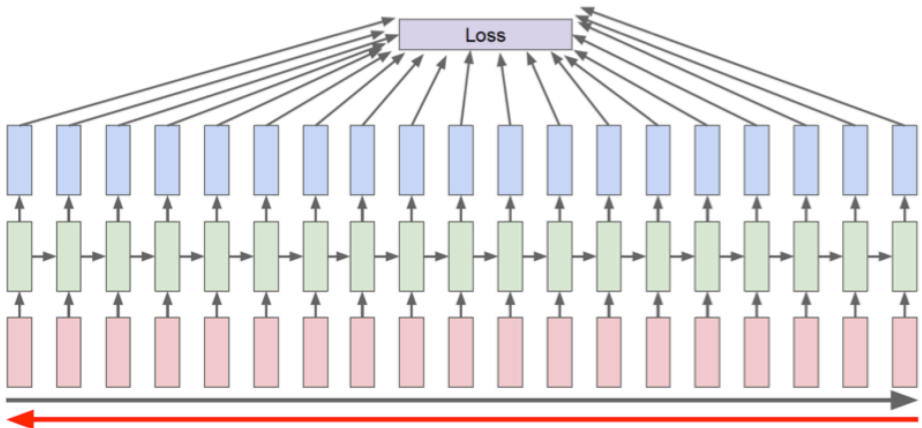
Exemple pour la classification :

- Erreur au pas de temps t : $\mathcal{L}_t(\mathbf{y}_t, \mathbf{y}_t^*)$ (entropie croisée)
- Erreur totale : $\mathcal{L}(\{\mathbf{y}_t\}, \{\mathbf{y}_t^*\}) = \sum_{t=1}^T \mathcal{L}_t(\mathbf{y}_t, \mathbf{y}_t^*)$



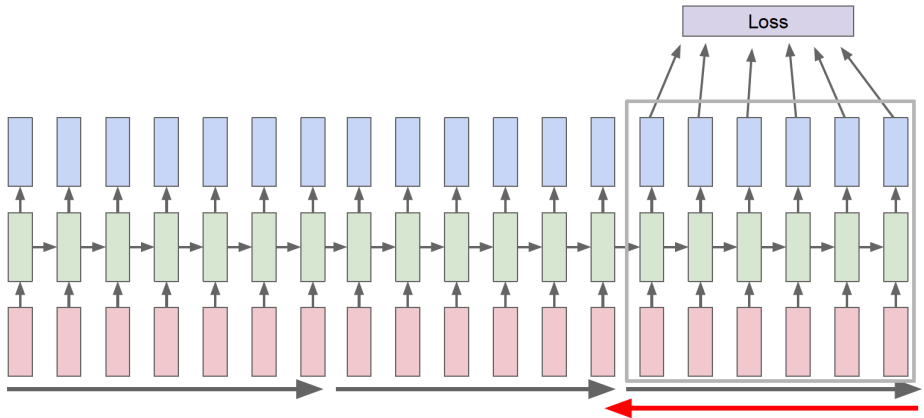
Back-propagation through time (BPTT)

Pour optimiser le modèle, on applique la rétropropagation dans le temps (WERBOS 1988) sur la vue "déroulée" (*unfolded*) du réseau récurrent pour l'écrire comme un long réseau entièrement connecté à poids partagés.



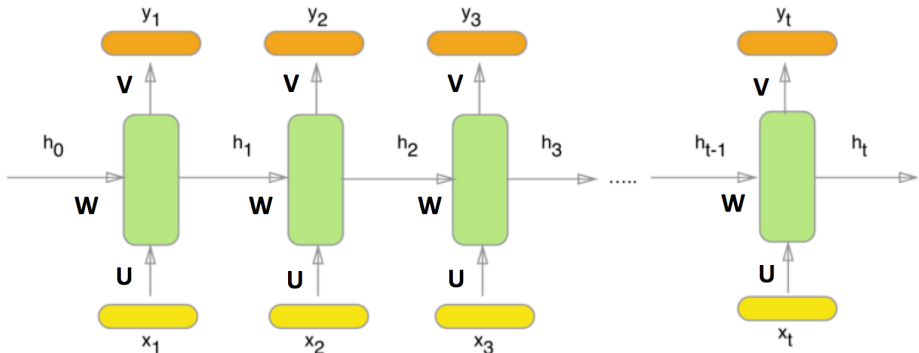
BPTT tronquée

Si T est grand, alors le réseau déroulé est très profond et la rétropropagation est coûteuse. On peut éviter de rétropropager sur tout le graphe en tronquant l'algorithme :



BPTT : principe

- BPTT : les poids sont mis à jour par descente de gradient, il faut donc calculer $\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}}$, $\frac{\partial \mathcal{L}_t}{\partial \mathbf{U}}$, $\frac{\partial \mathcal{L}_t}{\partial \mathbf{V}}$ (+biases)
- Dans le RNN déroulé, on applique la rétropropagation comme pour les réseaux classiques (*chain rule*) : **Unfolded RNN** : same spirit as back-prop with fully connected networks (*chain rule*)
 - **Différence** : les paramètres $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$ sont partagés pour tous les pas de temps, leur mise à jour est la moyenne des gradients pour chaque t .

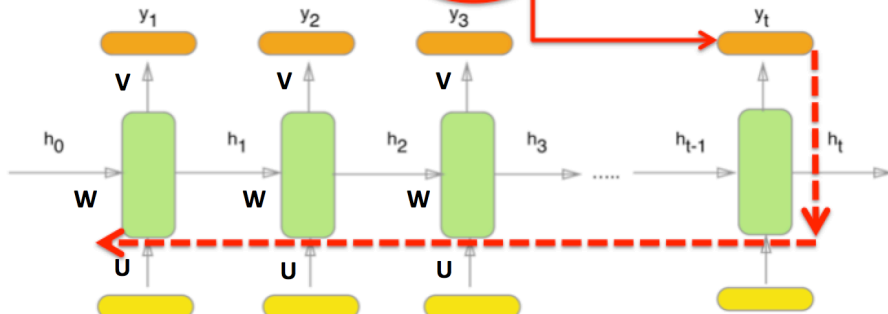


BPTT : calcul des gradients

- Les paramètres $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$ sont partagés dans le temps $\Rightarrow$ les gradients dépendent de tout l'historique passé
- Exemple pour $\mathbf{W}$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}$$

$$\frac{\partial \mathcal{L}_t(\mathbf{y}_t, \mathbf{y}_t^*)}{\partial \mathbf{W}} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}}$$



BPTT : optimisation

- Méthodes de premier ordre (descente de gradient et variantes) : RMSProp, Adam
- Dans de rares cas, méthodes de second ordre (*Hessian-free*).

Difficultés

Comme la BPTT opère sur le réseau déroulé, on optimise un réseau très profond : forts risques de gradients évanescents !

Ces risques sont empirés par l'utilisation des fonctions d'activation non-linéaire classiques, comme $\tanh$ ou sigmoïde...

Bibliographie I

-  ELMAN, Jeffrey L. (1990). "Finding Structure in Time". In : *Cognitive Science* 14.2, p. 179-211. ISSN : 1551-6709. DOI : 10.1207/s15516709cog1402_1. URL : https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1402_1 (visité le 16/05/2023).
-  HAMMER, Barbara (1^{er} mars 2000). "On the Approximation Capability of Recurrent Neural Networks". In : *Neurocomputing* 31.1, p. 107-123. ISSN : 0925-2312. DOI : 10.1016/S0925-2312(99)00174-5. URL : <https://www.sciencedirect.com/science/article/pii/S0925231299001745> (visité le 16/05/2023).
-  SIEGELMANN, H. T. et E. D. SONTAG (1^{er} fév. 1995). "On the Computational Power of Neural Nets". In : *Journal of Computer and System Sciences* 50.1, p. 132-150. ISSN : 0022-0000. DOI : 10.1006/jcss.1995.1013. URL : <https://www.sciencedirect.com/science/article/pii/S0022000085710136> (visité le 16/05/2023).
-  WERBOS, Paul J. (1^{er} jan. 1988). "Generalization of Backpropagation with Application to a Recurrent Gas Market Model". In : *Neural Networks* 1.4, p. 339-356. ISSN : 0893-6080. DOI : 10.1016/0893-6080(88)90007-X. URL : <https://www.sciencedirect.com/science/article/pii/089360808890007X> (visité le 16/05/2023).